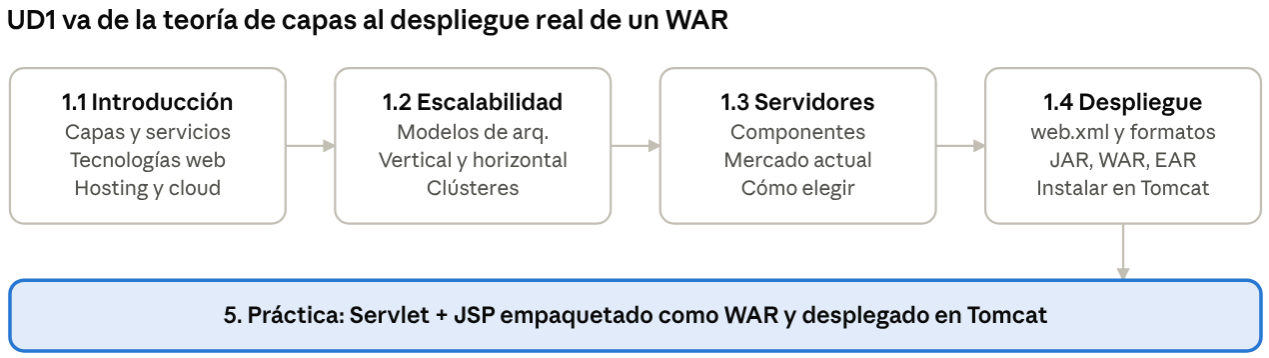
# Guía de la UD1: Implementación de Arquitecturas Web

*MP_0614 Despliegue de Aplicaciones Web · RA1: Implementa arquitecturas web analizando y aplicando criterios de funcionalidad*

Esta guía reúne y desarrolla los cuatro bloques de la unidad (1.1 Introducción, 1.2 Arquitecturas escalables y tolerantes a fallos, 1.3 Servidores de aplicaciones, 1.4 Descriptores de despliegue y distribución) y el ejemplo práctico de Servlet + JSP. Al final encontrarás tablas resumen, errores frecuentes y preguntas de repaso.




mapa de la unidad · 4 bloques + práctica

Empieza por este mapa: cada bloque construye sobre el anterior y la práctica final los junta todos.

## 1. Introducción a las arquitecturas web (UD 1.1)

### 1.1 La World Wide Web como modelo de programación

La arquitectura de la WWW ofrece un modelo de programación potente y flexible: las aplicaciones y el contenido se presentan en **formatos de datos estándar** (HTML, XML, etc.) y se localizan y visualizan mediante aplicaciones llamadas **navegadores web**. Esos estándares definen gran parte de los mecanismos necesarios para construir un entorno de aplicaciones de propósito general, tanto para los usuarios que consumen servicios como para los desarrolladores que los crean.

### 1.2 Aspectos clave de una arquitectura web

1. **Escalabilidad:** capacidad de crecer (más usuarios, más carga) sin rediseñar el sistema.
2. **Separación de responsabilidades:** cada parte del sistema (presentación, lógica, datos) tiene una función distinta y no invade la de otras.

3. **Portabilidad:** la aplicación funciona en distintos sistemas operativos, navegadores y dispositivos.
4. **Uso de componentes en servicios de infraestructura:** reutilizar piezas ya existentes (servidores, contenedores, bases de datos) en lugar de reinventarlas.
5. **Gestión de sesiones de usuario:** mantener el estado de cada usuario entre peticiones HTTP (que por sí mismo no guarda estado).
6. **Empleo de patrones de diseño:** soluciones probadas a problemas recurrentes (por ejemplo, MVC).

## 1.3 Un sistema de 3 capas

La arquitectura web se describe como un modelo de tres capas:

| Capa | Función | Ejemplos |
|---|---|---|
| **Capa de acceso a datos** | Almacena toda la información que gestiona el servicio web | MySQL, PostgreSQL |
| **Capa del servidor de aplicaciones web** | Ejecuta la aplicación y atiende las peticiones | Apache, Tomcat, Resin |
| **Capa del servicio web del cliente** | Donde el usuario final accede mediante un navegador | Firefox, Internet Explorer, Opera |

## 1.4 Los tres elementos de un servicio web

El esquema de funcionamiento de los servicios web requiere tres elementos:

- **Proveedor de servicio:** entidad que diseña, desarrolla e implementa el servicio y lo pone a disposición, dentro de la organización o públicamente.
- **Consumidor del servicio:** entidad que accede al servicio para utilizar lo que ofrece.
- **Agente del servicio:** enlace entre proveedor y consumidor; publica el servicio y facilita su búsqueda y localización.

## 1.5 Evolución de los servicios web

La evolución del uso de servicios web en las empresas está ligada al desarrollo de Internet como proveedor de servicios. Los factores que lo han impulsado son:

- **Más contenido dinámico:** un servicio web debe combinar contenido de fuentes distintas y entregarlo al instante.
- **Mayor ancho de banda:** permite entregar vídeo, audio y otros contenidos pesados.
- **Almacenamiento más barato:** facilita manejar cantidades masivas de datos.

- **Éxito de la computación extendida:** cientos de millones de teléfonos y tabletas; los ordenadores ya no son el dispositivo más común, así que el servicio debe atender a cualquier dispositivo, plataforma y navegador.

## 1.6 Nuevos objetivos tecnológicos

En este entorno surgen tecnologías orientadas a: transformar el software de escritorio en aplicaciones web, separar las hojas de estilo del contenido, potenciar el trabajo colaborativo y las redes sociales, y dar a los usuarios control sobre la gestión de su información.

## 1.7 Tecnologías web (cliente y servidor)

| Tecnología | Dónde se ejecuta | Descripción |
|---|---|---|
| **ASP** (Active Server Pages) | Servidor (IIS) | Lenguaje de Microsoft con páginas activas que generan resultados mostrados luego en el navegador |
| **CGI** (Common Gateway Interface) | Servidor | Uno de los estándares más antiguos para pasar información de una página a un servidor web (bases de datos, buscadores, formularios, correo automático). Rutinas en lenguajes interpretados (Perl) o compilados (C) |
| **CSS** | Cliente | Da formato a las páginas, separando el contenido de su presentación |
| **Java** | Cliente (applets) y servidor (Servlets/JSP) | Lenguaje y plataforma de Sun Microsystems, independiente del sistema operativo, navegador o dispositivo. En el cliente se ejecuta mediante *applets* gracias a la JVM del navegador |
| **PHP** | Servidor | Similar a ASP; accede a bases de datos como MySQL y sirve para páginas dinámicas complejas |

| Tecnología | Dónde se ejecuta | Descripción |
|---|---|---|
| **JavaScript** | Cliente | Lenguaje interpretado, útil para tareas interactivas; más compatible que VBScript |
| **VBScript** | Cliente o servidor | Respuesta de Microsoft a JavaScript, orientada a sitios vistos solo en Internet Explorer |

> **Nota de actualidad:** los *applets* y VBScript están hoy prácticamente abandonados (los navegadores modernos ya no los soportan). Se mantienen en el temario por su valor histórico y porque pueden aparecer en examen.

## 1.8 Hosting

**Hosting** (alojamiento web) es el servicio que proporciona a los usuarios de Internet un sistema para almacenar contenido accesible a través de la web: todos los archivos que forman un sitio, además de permitir copias de seguridad.

| Tipo | Ventajas | Desventajas |
|---|---|---|
| **Gratuito** | Útil para pruebas y aprendizaje; entorno de desarrollo sin coste. Suele incluir FTP, correo y soporte PHP | Poca potencia y memoria, graves limitaciones de recursos, servicio de muy baja calidad |
| **Compartido** | Económico (unos 4,50 €/mes); con cuentas de calidad soporta mucho tráfico; disponibilidad garantizada del 99,9 %. Lo usa ~95 % de usuarios y pequeñas empresas | Algunos proveedores saturan clientes y recursos: lentitud, inestabilidad, problemas con picos de tráfico |
| **VPS** (Virtual Private Server) | El cliente tiene acceso exclusivo a una máquina virtual; se adapta rápido a la demanda sin verse afectado por picos | Precio superior (>20-30 €/mes); los planes básicos rinden menos que un buen compartido; requiere más experiencia técnica |
| **Dedicado** | Servidor completo para aplicaciones grandes y tráfico muy elevado (lo usan los ISP para alojar páginas de sus clientes) | Coste alto; el tipo de servidor suele venir determinado por el sistema operativo instalado |

## 1.9 Cloud hosting

El alojamiento en la nube está pensado para sitios y aplicaciones que necesitan **alta disponibilidad** y no pueden permitirse interrupciones. Los términos *Cloud System*, *Cloud Software* y *Cloud Hosting* indican que se ejecutan en recursos de la nube o hacen uso de ellos.

**Tipos de cloud:**

- **Pública:** el proveedor ofrece sus recursos abiertamente a cualquier entidad, de particulares a grandes corporaciones (AWS, Microsoft Azure).
- **Privada:** se implementa y gestiona para una única entidad, con alto nivel de complejidad y centralización de recursos.
- **Híbrida:** combina las dos; algunos servicios e información son públicos y otros privados, lo que aporta escalabilidad.

**Modelos de servicio:**

- **SaaS** (Software as a Service): el usuario consume software completo basado en web (Dropbox, Gmail).

- **IaaS** (Infrastructure as a Service): el proveedor ofrece capacidad bruta de almacenamiento y procesamiento (servidores, balanceadores de carga) y el usuario construye todo lo demás desde cero.
- **PaaS** (Platform as a Service): el proveedor ofrece herramientas y utilidades para construir aplicaciones (bases de datos, entornos de ejecución, servidores web y de aplicaciones).

**¿Hosting tradicional o cloud?** Antes, las aplicaciones vivían en una o varias máquinas físicas, expuestas a daños y pérdida parcial o total de datos, con recursos limitados. El cloud usa **servidores virtuales interconectados**: si una máquina falla, otra arranca de inmediato sin afectar al servicio. El grupo funciona como una sola máquina (de forma similar a RAID 0/1) y los recursos o el almacenamiento pueden ampliarse al instante.

# 2. Arquitecturas escalables y tolerantes a fallos (UD 1.2)

## 2.1 Arquitectura web en tres capas lógicas

| Capa | Responsabilidad |
|---|---|
| **Presentación** | Navegabilidad, validación de datos de entrada, formateo de datos de salida y presentación web. Es la capa que ve el usuario |
| **Negocio** | Recibe las solicitudes de los usuarios, envía las respuestas y verifica que se cumplan las reglas establecidas |
| **Acceso a datos** | Gestores de datos específicos que almacenan, estructuran y recuperan los datos solicitados por la capa de negocio |

En Java, el esquema típico es: **JSP/HTML** (presentación) → **Servlets** (negocio) → **JDBC/base de datos** (acceso a datos).

## 2.2 Modelos de arquitectura web

1. **P2P (peer to peer):** red descentralizada y distribuida. Los nodos (*peers*) actúan a la vez como proveedores y clientes de recursos, aportando parte de sí mismos para otros nodos, **sin servidores centralizados**.
2. **Cliente-servidor:** aplicación distribuida que reparte la carga entre proveedores de recursos o servicios (servidores) y solicitantes (clientes) a través de una red. El navegador solicita una página indicando su URL; el servidor recibe la solicitud, busca la

información y la envía.

3. **Arquitectura de 2 niveles:** cliente-servidor en la que el servidor es versátil y responde directamente a todas las solicitudes del cliente.
4. **Arquitectura de 3 niveles:** hay un nivel intermedio entre el cliente y el servidor de datos. El cliente solicita recursos; el **servidor de aplicaciones** (*middleware*) los proporciona; el **servidor de datos** suministra los datos que pide el servidor de aplicaciones. Cada servidor se especializa en una tarea.
5. **Modelos con servidor de aplicaciones:** el servidor de aplicaciones contiene código que se ejecuta y envía el resultado al cliente **a través del servidor web**. Ambos servidores pueden estar en el mismo equipo. Lo habitual es que se conecte a una base de datos (en el mismo host o en otro). Si el servidor de aplicaciones es externo, el servidor web trabaja con menos carga porque funciona de forma independiente.

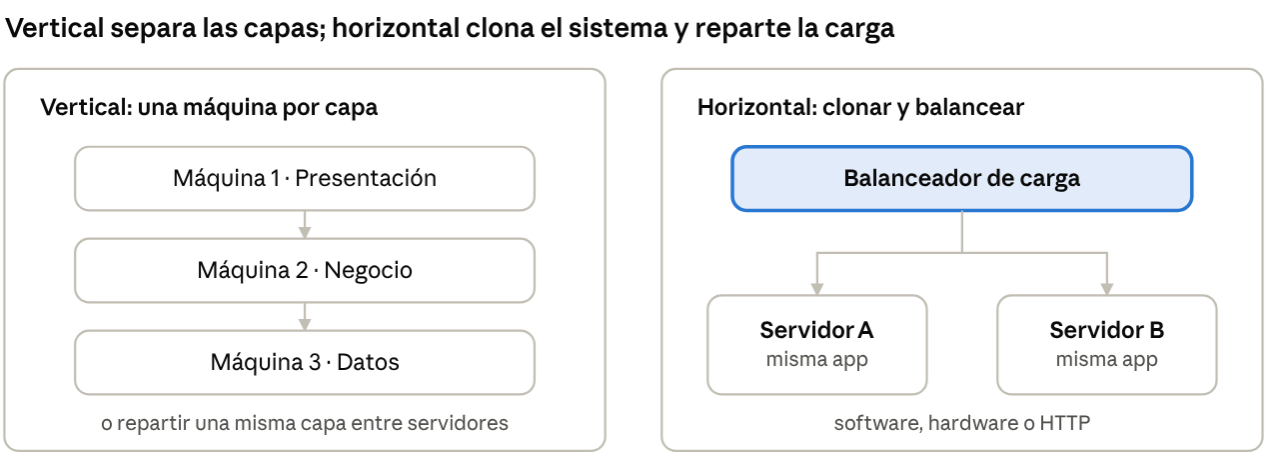
## 2.3 Escalabilidad

El número de clientes que solicitan un servicio varía según el momento, de ahí la importancia de escalar.

**Escalabilidad vertical (según el temario).** La separación lógica en capas se implementa para permitir una separación física: una máquina por cada capa, o repartir los elementos de una misma capa entre distintos servidores.

**Escalabilidad horizontal.** El sistema se clona en otra máquina de características similares y la carga se reparte mediante un dispositivo externo:

- **Balanceador de carga por software.**
- **Balanceador de carga por hardware:** dispositivo que redirige la solicitud HTTP de un usuario a la máquina adecuada.
- **Balanceador de carga hardware HTTP:** examina el paquete HTTP y mantiene la relación usuario-servidor (afinidad de sesión).




escalabilidad vertical frente a horizontal

Regla mnemotécnica del temario: vertical = separar por capas, horizontal = clonar la máquina y repartir con un balanceador.

> **Ojo con la terminología.** En muchos manuales "escalar verticalmente" significa añadir más CPU/RAM a una misma máquina (*scale up*), y "horizontalmente" añadir más máquinas (*scale out*). Para el examen, usa las definiciones del temario, pero conviene conocer ambas.

**Clúster de servidores.** Permite desplegar una aplicación normal de modo que su carga se distribuya entre los servidores del clúster **de forma transparente** para el usuario y para el administrador. Mediante **replicación de sesión**, cualquier máquina que atienda una petición HTTP tiene acceso a la sesión del usuario. Hay dos grandes tipos:

- **Clústeres de alta disponibilidad:** su objetivo es garantizar un servicio ininterrumpido.
- **Clústeres con equilibrio de carga:** distribuyen el tráfico entrante entre varios servidores con las mismas aplicaciones.

## 2.4 Tolerancia a fallos

Una arquitectura tolera fallos cuando el sistema sigue funcionando aunque falle un componente. Se consigue con **redundancia** (varias instancias), **balanceo de carga**, **replicación de sesión** y **recuperación ante fallos** (otra máquina asume el trabajo). Es la razón por la que clústeres y cloud hosting ofrecen alta disponibilidad.

# 3. Servidores de aplicaciones (UD 1.3)

## 3.1 De los servidores web a los servidores de aplicaciones

En la primera actividad se instaló **XAMPP**, un paquete multiplataforma, gratuito y de código abierto que simplifica montar un servidor web local. Incluye:

- **Apache:** popular servidor web de código abierto, conocido por su flexibilidad y extensibilidad.
- **MySQL:** sistema gestor de bases de datos (SGBD) de código abierto.
- **WordPress** (instalado sobre ello): CMS de código abierto para crear sitios web.

El siguiente paso es el **servidor de aplicaciones**: un servidor capaz de ejecutar ciertas aplicaciones, al que nos conectamos por red desde una aplicación cliente.

- **A nivel de servidor,** las plataformas ofrecen una **API** que facilita crear aplicaciones a través de varias interfaces (sobre todo web), desvinculando en gran medida el desarrollo del sistema operativo.
- **En el lado del usuario** hace falta una **aplicación cliente** que se conecte al servidor mediante un mecanismo de comunicación, normalmente una red **TCP/IP**; solicita la ejecución de la aplicación y recibe el resultado.
- La comunicación es **interactiva**: durante la ejecución el usuario intercambia datos con el servidor y viceversa.
- Entre cliente y servidor se establecen **mecanismos de seguridad** (autenticación del usuario y derechos de ejecución).

Existen plataformas propietarias y de código abierto. Tras el éxito de Java aparecieron los servidores **Java EE**: propietarios (WebLogic de Oracle, antes BEA Systems; WebSphere de IBM; EAServer de Sybase) y de código abierto (JOnAS, JBoss, Geronimo, TomEE de Apache).

## 3.2 ¿Por qué usar un servidor de aplicaciones?

1. **Alta disponibilidad:** diseñados para funcionar 24/7, con redundancia, equilibrio de carga, recuperación ante fallos y medidas de seguridad.
2. **Escalabilidad:** si crecen los requisitos se puede aumentar la potencia de los servidores o su número.
3. **Gestión y mantenimiento centralizados:** actualizar y mantener una aplicación en el servidor es más sencillo que si está "dispersa" en el ordenador de cada usuario.

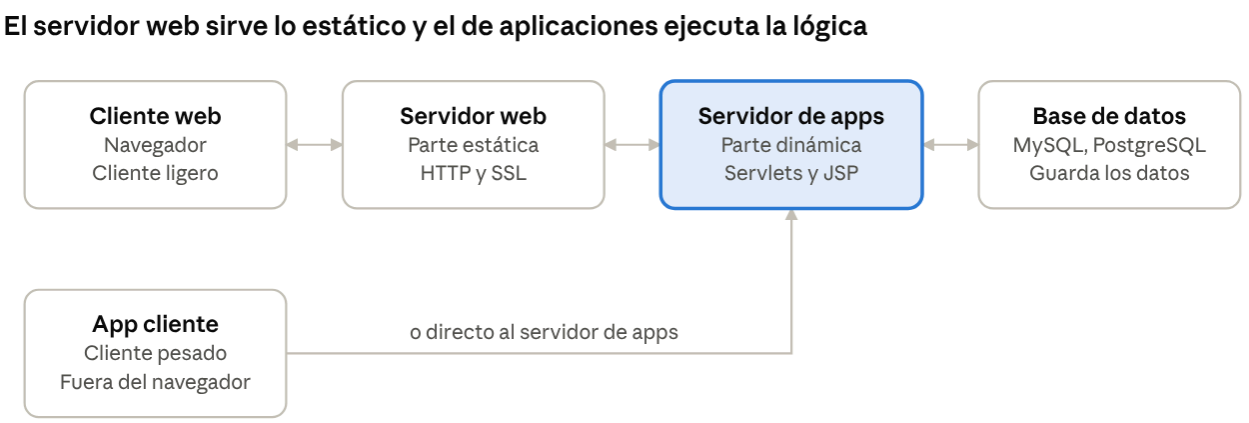
## 3.3 ¿Y por qué no? Aplicaciones que conviene instalar localmente

No todas las aplicaciones encajan en el modelo cliente-servidor; el **administrador del sistema** decide la arquitectura operativa. Suelen instalarse en local:

- **Diseño gráfico y edición de vídeo** (Photoshop, Premiere): exigen mucha potencia.
- **Herramientas de desarrollo** (Visual Studio, Android Studio): los desarrolladores necesitan acceso inmediato a recursos locales.
- **Aplicaciones muy especializadas** para pocos usuarios: no requieren recursos del servidor ni conectividad constante.
- **Análisis de datos** (Excel, R): grandes conjuntos de datos y cálculo intensivo en el equipo del usuario.
- **Diseño 3D y CAD** (AutoCAD): requieren potencia de procesamiento significativa.

## 3.4 Componentes de la arquitectura

| N.º | Componente | Descripción |
|---|---|---|
| 1 | **Cliente web** | Navegador que interactúa con el servidor web vía HTTP; recibe HTML/XML y puede ejecutar applets y JavaScript. Se le llama **cliente ligero** |
| 2 | **Aplicación cliente** | Programa que se ejecuta en el equipo cliente, independiente del navegador; puede hablar con el servidor web o directamente con el de aplicaciones. Es un **cliente pesado** (p. ej. una app Java con interfaz gráfica o de consola) |
| 3 | **Servidor web** | También "contenedor web" o "motor web". Es la parte visible del servidor de aplicaciones; se comunica por HTTP o protocolos seguros (SSL). Gestiona la parte **estática** (estructura de las páginas) |
| 4 | **Servidor de aplicaciones** | Conjunto de programas que ejecutan la aplicación y ofrecen servicios empresariales. Responsable de la parte **dinámica** |
| 5 | **Base de datos** | Almacén de datos de la aplicación; puede servir solo al servidor o también a programas cliente |




componentes de la arquitectura · 5 elementos

El cliente ligero pasa siempre por el servidor web; el cliente pesado puede saltarse ese paso y hablar con el servidor de aplicaciones.

## 3.5 Estado actual del mercado

Hace una década dominaban soluciones monolíticas y propietarias; hoy la tendencia es hacia soluciones **modulares y de código abierto**.

**Servidores web:** Apache (dominante durante años), Nginx (alto rendimiento y eficiencia, gana terreno por manejar muchas conexiones simultáneas con poco consumo), IIS (Microsoft, común en infraestructuras Windows), y otros como LiteSpeed y Caddy. Tendencia: servidores web en la nube (AWS, Azure, Google Cloud).

**Servidores de aplicaciones:** WildFly (Java EE, antes JBoss), Tomcat (referente muy ligero para Java), GlassFish (Java EE, hoy impulsado por la comunidad), Oracle WebLogic e IBM WebSphere. Tendencias: **contenedores** (Docker, Kubernetes), **microservicios** (alta escalabilidad y flexibilidad) y preferencia por el **código abierto**.

## 3.6 Cómo elegir un servidor

- **Propósito:** si se sirve sobre todo contenido estático (HTML, CSS, imágenes), basta un servidor web optimizado; si hay procesamiento en servidor, bases de datos o contenido dinámico, hace falta un servidor de aplicaciones robusto.
- **Escalabilidad y rendimiento:** balanceo de carga y mecanismos de **caché** para reducir carga y mejorar tiempos de respuesta.
- **Coste y soporte:** licencias (propietario frente a código abierto), mantenimiento (actualizaciones, parches, posibles caídas) y soporte (comunidad o proveedor).
- **Personalización:** plugins o extensiones y capacidad de adaptación a necesidades concretas.

- **Seguridad:** firewalls, **SSL/TLS** para cifrar la transmisión (sobre todo con información confidencial) y actualizaciones regulares contra vulnerabilidades conocidas.
- **Compatibilidad:** integración sencilla con otras herramientas, bases de datos y sistemas.

# 4. Descriptores de despliegue y distribución (UD 1.4)

## 4.1 ¿Qué es un descriptor de despliegue?

Son archivos o configuraciones que definen **cómo debe implementarse y configurarse una aplicación web** en un entorno concreto (servidor web o contenedor de aplicaciones). Aportan la información necesaria para que la aplicación se instale y ejecute correctamente.

**Contenido habitual:**

1. **Configuración del servidor:** detalles del entorno de alojamiento (CPU, memoria, almacenamiento disponible).
2. **Recursos:** bases de datos, archivos y servicios externos que necesita la aplicación y cómo acceder a ellos.
3. **Variables de entorno:** conexiones a bases de datos, claves de API o datos confidenciales, protegiendo la información sensible.
4. **Escalabilidad:** cómo escalar para distintos niveles de tráfico (añadir servidores o instancias cuando crece la demanda).

## 4.2 Formatos de fichero para descriptores

- **XML:** muy utilizado; estructurado y legible por humanos.
- **YAML** (*YAML Ain't Markup Language*): formato de serialización fácil de usar, menos verboso que XML; muy usado en desarrollo actual.
- **JSON:** ligero, basado en texto, fácil de leer y escribir para personas y máquinas.

## 4.3 Ejemplo: `web.xml`

El descriptor estándar de una aplicación web Java EE es `WEB-INF/web.xml`:

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
         http://java.sun.com/xml/ns/javaee/web-app_3_0.xsd">
    <servlet>
        <servlet-name>CalcuServlet</servlet-name>
        <servlet-class>calculadora.CalcuServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>CalcuServlet</servlet-name>
        <url-pattern>CalcuServlet</url-pattern>
    </servlet-mapping>
    <session-config>
        <session-timeout>30</session-timeout>
    </session-config>
</web-app>
```

- `<servlet>` declara el servlet: un nombre lógico y la clase Java que lo implementa.
- `<servlet-mapping>` asocia ese servlet con una URL ( `url-pattern` ).
- `<session-config>` configura las sesiones; `session-timeout` son los minutos de inactividad antes de caducar (aquí, 30).

Desde Servlet 3.0 se puede sustituir parte de este XML por anotaciones como `@WebServlet`, y el `web.xml` pasa a ser opcional.

## 4.4 Tipos de empaquetado Java

| Formato | Significado | Uso |
|---|---|---|
| **JAR** | Java ARchive | Almacena aplicaciones o bibliotecas en Java. Es simplemente un ZIP con extensión `.jar` |
| **WAR** | Web Application Archive | Archivo JAR (con extensión `.war` ) para distribuir JSP, servlets, clases Java, archivos XML y contenido web estático (HTML). Juntos forman una aplicación web |
| **EAR** | Enterprise Application aRchive | Formato de Java EE que empaqueta varios módulos en un solo archivo y se despliega (con XML como descriptores) en un servidor de aplicaciones |

> En las diapositivas de la sección 5 el WAR aparece como "Web Application Resource"; el nombre estándar es **Web Application Archive**.

## 4.5 Estructura de una aplicación web Java EE

Eclipse genera esta estructura automáticamente al crear un **Dynamic Web Project**:

```
Directorio raíz (WebContent)   → páginas HTML, JSP, imágenes
└── WEB-INF
    ├── web.xml                → descriptor de despliegue
    ├── classes                → clases de la aplicación (servlets, modelo)
    └── lib                    → librerías adicionales (.jar)
```

- La carpeta raíz contiene los elementos estáticos y las páginas JSP.
- `WEB-INF` **no es accesible directamente desde el navegador**, por eso guarda la configuración y el código.
- Los servlets y demás clases compiladas van en `classes`.
- Los `.jar` que se añaden al proyecto (por ejemplo, el conector JDBC de MySQL) los coloca Eclipse en `lib`.

## 4.6 Ficheros WAR

Un WAR es un archivo comprimido que contiene toda la estructura de carpetas y archivos de una aplicación web Java EE. **Todos los servidores de aplicaciones Java EE lo admiten**, por lo que es el formato recomendado para distribuir e instalar.

**Cómo generarlo en Eclipse:** clic derecho sobre el proyecto → **Export** → **WAR file**.

## 4.7 Instalación en el servidor

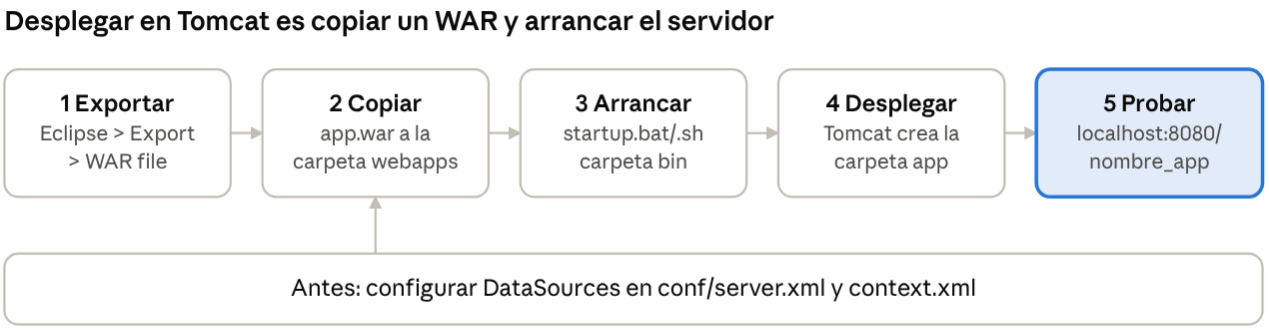
La instalación la realizan los **administradores del servidor**. El proceso es específico de cada servidor, pero siempre incluye dos pasos:

1. **Configuración de recursos:** el administrador configura los recursos que la aplicación necesita, como los **DataSources**. En Tomcat se puede hacer desde Eclipse o directamente en los archivos `server.xml` y `context.xml` de la carpeta `conf` del directorio de instalación.
2. **Instalación de la aplicación:** se instala el desplegable generado por el desarrollador (por ejemplo, un `.war`).

## 4.8 Despliegue en Tomcat paso a paso

1. Copiar el archivo `.war` en la carpeta **`webapps`** de Tomcat.

2. Arrancar Tomcat con `startup.bat` (Windows) desde la carpeta `bin`. En Linux/macOS el equivalente es `startup.sh`. Para detenerlo, `shutdown.bat` / `shutdown.sh`.
3. Comprobar que Tomcat ha **descomprimido** el `.war` y ha creado una carpeta con el nombre de la aplicación dentro de `webapps`.
4. Probar en el navegador: `http://localhost:8080/nombre_app`.




despliegue en Tomcat · 5 pasos

Los pasos 1 a 5 son del desarrollador; la configuración de recursos previa es tarea del administrador.

# 5. Práctica: Servlet + JSP en Eclipse con Tomcat

## 5.1 Qué es un Servlet

Un **Servlet** es una clase Java que se ejecuta en un contenedor (Tomcat) y atiende peticiones HTTP: recibe la petición, ejecuta la lógica de negocio y genera la respuesta (HTML, texto, JSON…). Encaja en la **capa de negocio**.

**Ciclo de vida:**

1. `init()`: se ejecuta una vez al cargar el servlet; inicializa recursos.
2. `service()` → `doGet()` / `doPost()`: se ejecuta en cada petición. `doGet` atiende peticiones GET (URL, enlaces) y `doPost` peticiones POST (formularios).
3. `destroy()`: se ejecuta al descargar el servlet o detener la aplicación; libera recursos.

## 5.2 La vista: formulario JSP

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"
pageEncoding="ISO-8859-1"%>
<%@ page import="java.util.Date"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="ISO-8859-1">
<title>Actividad 2</title>
</head>
<body>
    <form method="post" action="MyServlet">
        Nombre: <input type="text" name="username" id="username"/> <br/>
        <input type="submit" value="Vamos allá"/>
    </form>
</body>
</html>
```

Puntos clave:

- `<%@ page ... %>` es una **directiva JSP** que fija el lenguaje, el tipo de contenido y la codificación.
- `<%@ page import="java.util.Date"%>` importa una clase; aquí no se usa, así que puede eliminarse.
- `method="post"` envía los datos en el cuerpo de la petición, por lo que se ejecutará `doPost`.
- `action="MyServlet"` coincide con la URL declarada en `@WebServlet("/MyServlet")`.
- El atributo `name="username"` es la clave que el servlet usará con `getParameter`.

## 5.3 El controlador: el Servlet

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

@WebServlet("/MyServlet")
public class MyServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
```

```
    public MyServlet() { super(); }

    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException { }

    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        PrintWriter out = response.getWriter();
        String name = request.getParameter("username");
        System.out.println("La palabra " + name + " tiene " + name.length() +
" caracteres");
    }
}
```

**Explicación línea a línea:**

- `@WebServlet("/MyServlet")`: mapea la URL `/MyServlet` a esta clase (equivale a `<servlet>` + `<servlet-mapping>` en `web.xml`).
- `extends HttpServlet`: heredamos el comportamiento HTTP básico y sobrescribimos los métodos que nos interesan.
- `serialVersionUID`: identificador de versión de serialización que Eclipse genera por defecto; no afecta a la lógica.
- `HttpServletRequest request`: encapsula la petición (parámetros, cabeceras, cookies, sesión). `request.getParameter("username")` recupera el valor del campo del formulario.
- `HttpServletResponse response`: sirve para construir la respuesta (código de estado, tipo MIME con `setContentType`, flujo de salida con `getWriter()`).
- `System.out.println(...)`: escribe en la **consola del servidor** (la pestaña Console de Eclipse o el log de Tomcat), **no en el navegador**.

## 5.4 Puntos a mejorar del ejemplo

- **Respuesta vacía:** el servlet obtiene el `PrintWriter` pero no escribe nada, así que el navegador muestra una página en blanco. Para responder al usuario hay que escribir en `out`.
- **Posible `NullPointerException`:** si `getParameter` devuelve `null` (parámetro ausente), `name.length()` falla. Conviene comprobar `name != null`.
- **`doGet` vacío:** si se accede a `/MyServlet` escribiendo la URL, no ocurre nada visible.
- **Codificación:** el formulario usa ISO-8859-1; con tildes o eñ puede dar problemas. UTF-8 es lo recomendable.

Versión mejorada de `doPost`:

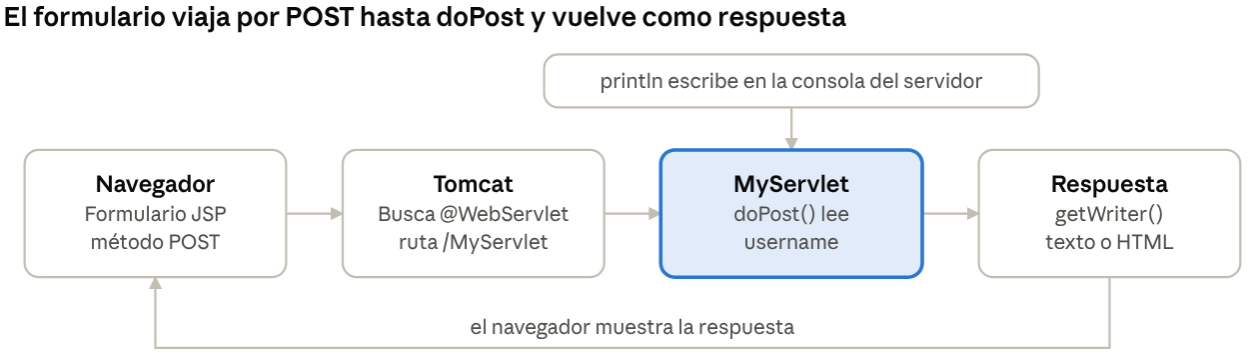
```
protected void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {
    request.setCharacterEncoding("UTF-8");
    response.setContentType("text/html; charset=UTF-8");
    PrintWriter out = response.getWriter();

    String name = request.getParameter("username");
    if (name == null || name.isEmpty()) {
        out.println("<h1>Debes introducir un nombre</h1>");
    } else {
        out.println("<h1>La palabra " + name + " tiene " + name.length() + "
caracteres</h1>");
    }
}
```

> **Compatibilidad:** los ejemplos usan el paquete `javax.servlet`, válido hasta Tomcat 9. A partir de **Tomcat 10** el espacio de nombres pasó a `jakarta.servlet`, y hay que cambiar los `import` en consecuencia.

## 5.5 Flujo completo de una petición

1. El usuario abre la página JSP y escribe su nombre.
2. Al pulsar el botón, el navegador envía una petición **POST** a `/MyServlet` con `username=...`.
3. Tomcat localiza el servlet mediante `@WebServlet` y llama a `doPost`.
4. El servlet lee el parámetro, procesa y responde.
5. El navegador muestra la respuesta.




ciclo de una petición POST al servlet

Si en el examen te preguntan dónde sale el mensaje de `println`, la respuesta es la consola del servidor.

### 5.6 Pasos prácticos en Eclipse

1. Crear un **Dynamic Web Project** (*File → New → Dynamic Web Project*) y elegir el runtime de Tomcat.
2. Crear el JSP en la carpeta `WebContent` (o `src/main/webapp`).
3. Crear el Servlet en `src` (*New → Servlet*).
4. Añadir Tomcat en la pestaña **Servers**, asociar el proyecto y arrancarlo (*Run on Server*).
5. Para distribuirlo: *clic derecho → Export → WAR file*, copiar el `.war` a `webapps` y arrancar Tomcat como se explicó en el apartado 4.8.

## 6. Resumen comparativo

| Concepto | Idea clave |
|---|---|
| Capas | Presentación, negocio y acceso a datos |
| Cliente ligero / pesado | Navegador frente a aplicación propia instalada |
| Servidor web | Parte estática, HTTP/HTTPS |
| Servidor de aplicaciones | Parte dinámica y servicios empresariales |
| Escalabilidad vertical (temario) | Separar capas en máquinas distintas |
| Escalabilidad horizontal | Clonar el sistema y balancear la carga |
| Clúster | Reparto transparente de carga con replicación de sesión |
| JAR / WAR / EAR | Biblioteca / aplicación web / aplicación empresarial con varios módulos |
| `web.xml` | Descriptor de despliegue en `WEB-INF` |
| Despliegue en Tomcat | Copiar `.war` en `webapps` y arrancar |

## 7. Errores frecuentes

- Confundir **servidor web** (estático) con **servidor de aplicaciones** (dinámico).
- Creer que `System.out.println` en un servlet muestra texto en el navegador.
- Poner archivos sensibles fuera de `WEB-INF`, donde sí son accesibles por URL.

- Olvidar que el `action` del formulario debe coincidir con el mapeo del servlet.
- Copiar el `.war` con Tomcat arrancado y no esperar al despliegue automático, o usar una carpeta antigua con el mismo nombre.
- Mezclar `javax.servlet` y `jakarta.servlet` según la versión de Tomcat.

## 8. Preguntas de repaso

1. Define las tres capas de una arquitectura web y cita una tecnología para cada una.
2. Diferencia P2P, cliente-servidor, 2 niveles y 3 niveles.
3. ¿Qué diferencia hay entre escalabilidad vertical y horizontal según el temario? ¿Qué es un balanceador HTTP?
4. ¿Qué es un clúster y qué papel juega la replicación de sesión?
5. Explica tres razones para usar un servidor de aplicaciones y tres tipos de aplicaciones que conviene instalar en local.
6. Compara hosting compartido, VPS y dedicado.
7. Diferencia SaaS, PaaS e IaaS y pon un ejemplo de cada uno.
8. ¿Qué es un descriptor de despliegue y qué contenidos puede incluir? ¿En qué formatos se escribe?
9. Describe la estructura de carpetas de un proyecto web dinámico y la función de `WEB-INF`.
10. Diferencia JAR, WAR y EAR.
11. Enumera los pasos para desplegar un `.war` en Tomcat.
12. En el ejemplo de la sección 5, ¿qué método del servlet se ejecuta y por qué? ¿Dónde aparece el mensaje de `System.out.println`?
13. Cita tres criterios para elegir un servidor y explica uno de ellos.